

NOVEMBER 4, 2025

MUSIC TRANSFER APP SYSTEM REQUIREMENTS DOCUMENT

DRAFT

TEAM SNOWMEN ❄️🧊🧊☁️

KORAL MOYA, MATHEW WATERS, TAYLOR SHEEER

TABLE OF CONTENTS

Introduction (MW)	2
Description Model (MW)	2
Class Diagram (KM)	4
Use Case Diagram (KM)	4
Use Case Scenarios (MW)	4
System Sequence Charts (KM)	14

INTRODUCTION (MW)

The purpose of this Requirements Document is to define the functional and non-functional requirements of the Music Transfer App. It outlines what the system will do, who the system's actors are, and how the system supports playlist management across multiple music streaming services. This document provides a clear overview of the system's scope and expected behavior for developers and stakeholders

This document includes a requirements overview, a class diagram for the anticipated major system objects, a use case diagram and complete use case scenarios, and system sequence charts documenting each major process. These components serve as the basis for all subsequent documentation during the design, development, and testing stages of the project.

DESCRIPTION MODEL (MW)

The Music Transfer App allows users to transfer, update, backup, and share playlists on various music streaming applications. Additionally, users socially connect with one another through follow options, messaging, and user engagement/management (blocking). This section outlines the system requirements according to Output, Input, Processes, Performance and Security.

1. OUTPUT REQUIREMENTS

The system must produce the following outputs:

1. Confirmation screens for actions such as transferring, saving, synchronizing, deleting, or removing playlists.
2. Transfer results summarizing matched, unmatched, or skipped tracks.
3. A public or private share link when a playlist is shared with another user.
4. System notifications for new messages, followers, playlist updates, and successful/failed operations.
5. Updated playlist views after synchronization or deletion.
6. Error messages when connections fail, track matches cannot be found, or a user is blocked.

2. INPUT REQUIREMENTS

The system must accept the following user inputs:

1. Login credentials and music service authorization tokens.
2. Selection of a playlist, target music service, or synchronization destination.

3. User actions such as Save Playlist, Remove Playlist, Share Playlist, Follow User, Block User, and Message User.
4. Message content typed by the user.
5. Confirmation responses for actions like deleting or blocking.
6. Manual track selections when resolving unmatched audio fingerprints.

3. PROCESS REQUIREMENTS

The system must perform the following internal processes:

1. Authenticate users and validate connections to external music services through APIs.
2. Transfer playlists between platforms using metadata matching and optional audio fingerprinting.
3. Synchronize playlists by comparing track lists and updating missing or changed songs.
4. Save and remove playlists from the user's personal library.
5. Share playlists by generating links or sending internal messages.
6. Manage social interactions: following, messaging, and blocking.
7. Store user data, follower lists, message histories, and playlist logs.
8. Handle exceptions such as failed transfers, missing tracks, duplicate saves, and restricted communication.

4. PERFORMANCE REQUIREMENTS

The system must meet the following performance expectations:

1. Playlist transfers and synchronizations should complete within 60 seconds for playlists containing up to 100 songs, and scale proportionally for larger playlists, provided external music service APIs are responding normally.
2. API requests to streaming platforms should respond within acceptable limits (typically under 3 seconds).
3. The system should support concurrent users without noticeable slowdown.
4. Messages between users should be delivered in real-time or near real-time.
5. The app should maintain smooth performance when handling large playlists (500+ songs).

5. SECURITY REQUIREMENTS

The system must meet the following security standards:

1. All user credentials and API tokens must be stored securely and never exposed in plain text. API requests are used to authenticate users, retrieve playlist and track metadata, create and delete playlists, synchronize track lists, and update user libraries on external music streaming platforms through secure OAuth-based access.
2. OAuth protocols must be used for authentication with external music services.
3. Private messages must be encrypted during transmission and storage.
4. Users must not be able to view private playlists unless explicitly shared.
5. Blocked users must be prevented from sending messages, viewing profiles, or following the blocking user.
6. Data access must follow the principle of least privilege, ensuring users can only modify their own playlists and data.

CLASS DIAGRAM (KM)

Class Diagram.vsd

USE CASE DIAGRAM (KM)

Use Case Diagram.vsd

USE CASE SCENARIOS (MW)

Use Case Scenario #1

Use Case name: Transfer Playlist

Primary Actor: User

Other Systems: Connected streaming services.

Description:

The system analyzes the selected playlist and matches each track using metadata such as track title, artist, and album, with optional audio fingerprinting used to identify tracks when metadata is missing, inconsistent, or unavailable across streaming services. Then it recreates that playlist on the profile on the streaming service you want to.

Preconditions:

1. User is logged into the Music Transfer App.
2. User has connected at least two music service accounts.
3. Source playlist exists and is accessible.

Postconditions:

- A new playlist is created on the target music service.
- System stores a log of the transfer and updates the user's playlist history.

Main Flow:

1. User selects the "Transfer Playlist" option from the app dashboard.
2. System displays a list of available music services and playlists.
3. User chooses the source music service and the playlist to transfer.
4. System prompts for a destination music service.
5. User selects the target platform.
6. System retrieves the tracks and metadata from the source playlist.
7. System performs track-matching using metadata (title, artists) and audio fingerprinting if needed.
8. System creates a new playlist on the target service.

9. System displays a confirmation message summarizing the transfer results.
10. User reviews and confirms completion.

Exceptions:

1. System Identifies unmatched Tracks
 - a. The system then presents the users with options of would the user like to manually search for it, to skip the song or retry. It then completes the playlist.
2. System requires the user to re-authenticate
 - a. The user will relogin and the system resumes the transfer.

Use Case Scenario# 2 — Connect Music Services

Use Case Name: Connect Music Services

Primary Actor: User

Other Systems: External music service APIs

Description: The user connects one or more music streaming services to the Music Transfer App by selecting a source service to transfer playlists from and connecting a destination service to receive transferred playlists

Preconditions:

1. User is logged in.
2. Music services are available via API.

Postconditions:

1. Connected music service is added to the user's account.
2. Authentication token is securely stored.

Main Flow:

1. User selects "Connect Music Services" from the app menu.
2. System displays a list of supported music platforms.
3. User selects a platform to connect (ex, Spotify).
4. System redirects user to the platform's authorization page.
5. User logs in and grants permission for playlist access.
6. Platform sends authorization confirmation back to the Music Transfer App.
7. System securely stores the access token.
8. System confirms successful connection and updates the user's service list.

Alternate / Exception Flows:

1. User Denies Access System displays a message that connection cannot be completed.
 - a. User returns to service list.
2. API Connection Fails
 - a. System shows error message and recommends retry.

Use Case Scenario# 3 — Share Playlist

Use Case Name: Share Playlist

Primary Actor: User

Description: User shares one of their playlists with another user through a generated link or internal messaging feature.

Preconditions:

1. User is logged in.
2. User has at least one accessible playlist.
3. Target user exists (if sharing internally).

Postconditions:

1. Playlist link or internal share message is successfully delivered.

Main Flow:

1. User navigates to one of their playlists.
2. User selects the “Share Playlist” option.
3. System provides sharing options: link, direct message, or social share.
4. User selects “Share via Direct Message.”
5. System prompts user to select a recipient from follower list or enter a username.
6. User selects the recipient.
7. System generates a shareable playlist link.
8. System sends the message to the recipient with the playlist link included.
9. System confirms successful share to the sender.

Alternate / Exception Flows:

A1 — User Chooses Public Link Instead System generates a public link. User copies the link or shares via external apps.

A2 — Recipient Has Blocked the User System denies sending and displays “Recipient unavailable.”

Use Case Scenario #4 — Save Playlist

Use Case Name: Save Playlist

Primary Actor: User

Description: The user saves a playlist (their own or someone else's) to their personal library for quick access.

Preconditions:

1. User is logged in.
2. Playlist exists and is viewable.

Postconditions:

1. Playlist appears in the user's "Saved Playlists."
2. "Remove Playlist" becomes available as an option.

Main Flow:

1. User views a playlist.
2. User selects the "Save Playlist" option.
3. System checks if playlist is already saved.
4. System adds playlist to user's saved library.
5. System confirms the playlist has been saved.

Alternate / Exception Flows:

1. Playlist Already Saved
 - a. System displays message "Playlist already in your library."

Use Case Scenario #5 — Remove Playlist

Use Case Name: Remove Playlist

Primary Actor: User

Description:

The user removes a playlist from their Saved Playlists library. The playlist is not deleted from the original platform — it is only removed from being bookmarked inside the Music Transfer App.

Preconditions:

1. User is logged in.

2. Playlist exists in the user's Saved Playlists.

Postconditions:

1. The playlist is removed from the user's saved library.
2. System updates the user's saved playlist list.

Main Flow:

1. User navigates to their Saved Playlists page.
2. User selects the playlist they want to remove.
3. User chooses the "Remove Playlist" option.
4. System prompts the user to confirm removal.
5. User confirms.
6. System removes the playlist from the user's Saved Playlists.
7. System displays a confirmation message.

Alternate / Exception Flows:

1. User Cancels the Removal
 - a. User selects "Cancel."
 - b. System closes the confirmation prompt and makes no changes.

Use Case Scenario 6 — Delete Playlist

Use Case Name: Delete Playlist

Primary Actor: User

Secondary System: Music Service API

Description:

The user deletes a playlist from their connected music service account entirely. This removes the playlist from the actual platform (ex, Spotify), not just from the app.

Preconditions:

1. User is logged in.
2. User has connected a music service.
3. Playlist exists on the music service and the user owns it.

Postconditions:

1. Playlist is permanently deleted on the music platform.
2. Any local references in the app are removed.

Main Flow:

1. User opens a playlist they created.
2. User selects the “Delete Playlist” option.
3. System requests confirmation due to permanent deletion.
4. User confirms.
5. System sends a delete request to the music service API.
6. API confirms deletion.
7. System removes any local references to the playlist.
8. System displays “Playlist successfully deleted.”

Alternate / Exception Flows:

1. API Returns Error
 - a. System notifies user that the playlist could not be deleted.
 - b. User may retry or cancel.

2. User Does Not Confirm Deletion

- a. Process ends with no changes made.

Use Case Scenario 7 — Synchronize Playlist

Use Case Name: Synchronize Playlist

Primary Actor: User

Secondary Systems: Connected Music Services

Description:

The user synchronizes two playlists across different platforms. The system compares songs on both services and updates the destination playlist with missing or newly added tracks.

Preconditions:

1. User is logged in.
2. At least two music services are connected.
3. Playlists selected for synchronization exist.

Postconditions:

1. Destination playlist is updated to match the source playlist.
2. System logs synchronization details.

Main Flow:

1. User opens a playlist they created.
2. User selects the “Synchronize Playlist” button
3. System fetches all tracks from both services.
4. System identifies songs missing in the target playlist.
5. System adds missing songs to the target playlist.

6. System confirms synchronization completion.
7. User reviews updated playlist.

Alternate / Exception Flows:

1. Some Tracks Cannot Be Matched
 - a. System lists unmatched tracks.
 - b. User may manually match or ignore them.
2. One Playlist Is Empty
 - a. System warns user and allows them to proceed or cancel.

Use Case Scenario 8 — Follow User

Use Case Name: Follow User

Primary Actor: User

Description:

The user follows another user within the Music Transfer App to see their shared playlists and updates.

Preconditions:

1. User is logged in.
2. Target user exists and is not blocked.

Postconditions:

1. Target user is added to the follower list.
2. User begins receiving updates from the followed user.

Main Flow:

1. User searches for or views another user's profile.
2. User selects the "Follow User" option.

3. System checks if user is already following the target.
4. If not, system adds the target user to the follower list.
5. System confirms the follow action.

Alternate / Exception Flows:

1. User Already Follows Target User
 - a. System displays: “You already follow this user.”
2. User Has Blocked Target User
 - a. System prompts the user to unblock before following.

Use Case Scenario 9 — Message User

Use Case Name: Message User

Primary Actor: User

Description:

The user sends a message to another user through the app’s internal messaging system.

Preconditions:

1. User is logged in.
2. Recipient exists.
3. Sender has not been blocked by the recipient.

Postconditions:

1. Message is stored and delivered to the recipient.
2. Recipient receives notification.

Main Flow:

1. User navigates to another user's profile.
2. User selects "Message User."
3. System displays a message input box.
4. User types a message and selects "Send."
5. System stores the message in the database.
6. System delivers message to the recipient.
7. System displays "Message Sent."

Alternate / Exception Flows:

1. Recipient Has Blocked Sender
 - a. System rejects the message and displays: "Cannot send message."
2. User Leaves Message Box Empty
 - a. System prompts "Message cannot be empty."

Use Case Scenario 10 — Block User

Use Case Name: Block User

Primary Actor: User

Description:

The user blocks another user, preventing messages, follows, or interactions.

Preconditions:

1. User is logged in.
2. Target user exists.

Postconditions:

1. Target user is added to the blocked list.
2. Target user can no longer follow or message the blocker.
3. Previously shared private playlists and direct message links between the blocker and blocked user are revoked and become inaccessible, while public playlist links remain

accessible unless manually removed by the playlist owner.

Main Flow:

1. User opens the target user's profile.
2. User selects "Block User."
3. System asks for confirmation.
4. User confirms.
5. System adds target to the blocked list.
6. System removes existing message ability or follower relationship.
7. System confirms: "User has been blocked."

Alternate / Exception Flows:

1. User Cancels Blocking
 - a. No changes are made.

SYSTEM SEQUENCE CHARTS (KM)

System Sequence Diagram(1).vsdx